CLASSIFICATION • MODEL 01

K-Nearest Neighbors (KNN)

From two-feature intuition to GridSearchCV

01 MEASURE

Distance between examples

02 SELECT

K closest neighbors

03 VOTE

Predict the class

Trainer presentation • Iris dataset • 5-fold stratified CV

A classification problem with two input features

Predict whether a new student passes or fails.

x_1	x_2	y
Hours studied	Mock score	Pass / Fail

New student

- Hours = 4.6
- Score = 66
- Result = ?

Question: Which labeled students are most similar to (4.6, 66)?

Our labeled training dataset

Each row is one known example with two numeric inputs.

Student	Hours (X_1)	Score (X_2)	Result
A	1	20	Fail
B	2	30	Fail
C	3	50	Fail
D	4	60	Pass
E	5	70	Pass
F	6	80	Pass
G	7	85	Pass
H	4	45	Fail

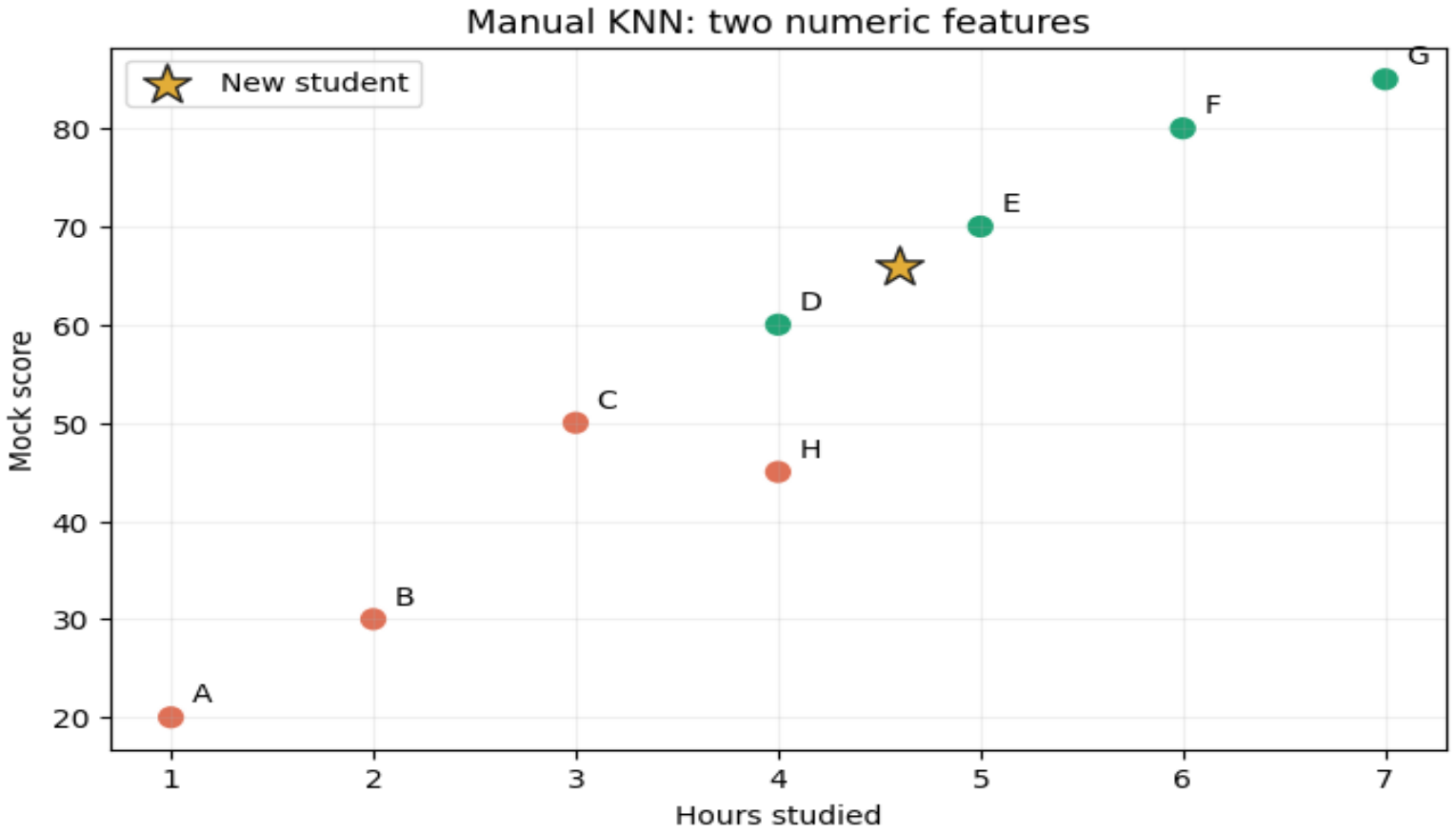
NEW STUDENT

(4.6, 66)

Unknown class

Plot two features on X and Y axes

Distance on this graph is the foundation of KNN.



- Every student becomes a point (X_1, X_2).
- Colors show known class labels.
- The star is the new, unlabeled student.
- KNN finds nearby labeled points.

Euclidean distance: How close are two points?

$$d = \sqrt{[(x_1,new - x_1,train)^2 + (x_2,new - x_2,train)^2]}$$

WORKED EXAMPLE: STUDENT E

New: (4.6, 66)

E: (5, 70)

Distance ≈ 4.02

$$d(E) = \sqrt{[(4.6 - 5)^2 + (66 - 70)^2]} = \sqrt{(0.16 + 16)} = \sqrt{16.16} \approx 4.02$$

Calculate distances to every training record

Sorted from the closest neighbor to the farthest.

Rank	Student	Distance	Result
1	**E**	**4.02**	**Pass**
2	**D**	**6.03**	**Pass**
3	**F**	**14.07**	**Pass**
4	C	16.08	Fail
5	G	19.15	Pass
6	H	21.01	Fail
7	B	36.09	Fail
8	A	46.14	Fail

K = 3

Use the top three rows

K=3: Nearest-neighbor majority voting

STUDENT E

PASS

Distance 4.02

STUDENT D

PASS

Distance 6.03

STUDENT F

PASS

Distance 14.07

3 Pass • 0 Fail → PREDICTION: PASS

What does changing K do?

K	Nearest student IDs	Pass votes	Fail votes	Prediction
1	E	1	0	Pass
3	E, D, F	3	0	Pass
5	E, D, F, C, G	4	1	Pass
7	E, D, F, C, G, H, B	4	3	Pass

Notice: changing K can change predictions, but it does not change this example's result.

Small K vs large K: Bias–variance tradeoff

SMALL K

Sensitive to local noise

- Detailed / jagged boundaries
- Higher variance
- Can overfit training points

LARGE K

Smooths over local patterns

- Simpler / smoother boundaries
- Higher bias if K is too large
- Can underfit

Choose K using cross-validation — not by guessing.

Class voting: Equal vs distance-weighted

A conceptual example in which the two methods disagree.

Neighbor	Distance	Class	1 / distance
1	2	Fail	0.500
2	5	Pass	0.200
3	8	Pass	0.125

Uniform → PASS

Distance-weighted → FAIL

UNIFORM VOTING

2 Pass vs 1 Fail

WEIGHTED VOTING

Fail: 0.500

Pass: 0.325

Why KNN generally needs feature scaling

Large numerical ranges can dominate the distance calculation.

Feature	Example difference	Squared contribution
Study hours	1 hour	$1^2 = 1$
Mock score	10 marks	$10^2 = 100$

The mock score contributes 100× as much to squared distance — solely due to its scale.

StandardScaler and the correct pipeline

STANDARDIZATION

$$z = (x - \mu_{train}) / \sigma_{train}$$

DURING CV

Fit the scaler only on each fold’s training partition.

PYTHON

```
Pipeline([(“scaler”, StandardScaler()), (“knn”, KNeighborsClassifier())])
```

Euclidean vs Manhattan distance

Both are supported by KNN; each defines “nearby” differently.

EUCLIDEAN • P = 2

$$\sqrt{(\Delta X_1^{\,2} + \Delta X_2^{\,2})}$$

New → E: $\sqrt{(0.4^2 + 4^2)}$

≈ 4.02

MANHATTAN • P = 1

$$|\Delta X_1| + |\Delta X_2|$$

New → E: 0.4 + 4

= 4.40

Both distances are scale-sensitive; tune the metric inside CV.

What exactly happens in fit() and predict()?

.FIT(X, Y)

Stores labeled examples

Prepares neighbor search

→

.PREDICT(NEW X)

Transforms new features

Finds nearest neighbors

→

VOTING

Aggregates neighbor labels

Returns predicted class

Unlike linear or logistic regression, KNN does not learn slope and intercept coefficients.

KNN: advantages, limitations, and use cases

Where KNN works well

- Simple to understand
- Non-linear decision boundaries
- Useful as a strong baseline
- Works with multiple classes

Watch-outs

- Sensitive to scaling and noise
- Prediction may be expensive
- Irrelevant features distort distance
- Distance degrades in high dimensions

The standard dataset: Iris

Built into scikit-learn; no download, Kaggle account, or CSV required.

150	3	4
Flower records	Species/classes	Original input features

FOR THIS EXERCISE WE SELECT 2 FEATURES

- **Petal length (cm)**
- **Petal width (cm)**

Target classes: setosa, versicolor, virginica

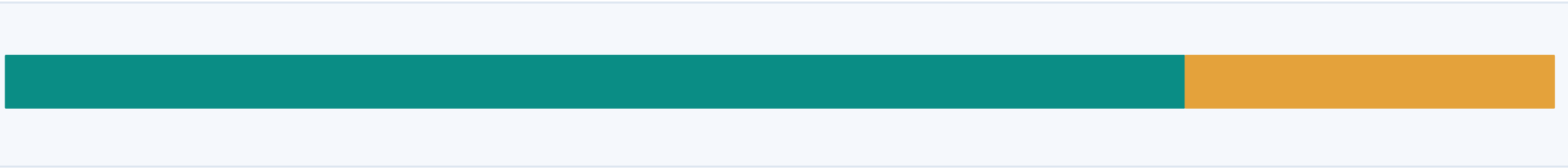
Visualize Iris: petal length and width

Can you spot where classes overlap?



- Setosa is clearly separated.
- Versicolor and virginica overlap.
- KNN predicts using nearby labeled flowers.

Train/test split: Keep the test set untouched



75% TRAINING • 112 FLOWERS

25% TEST • 38

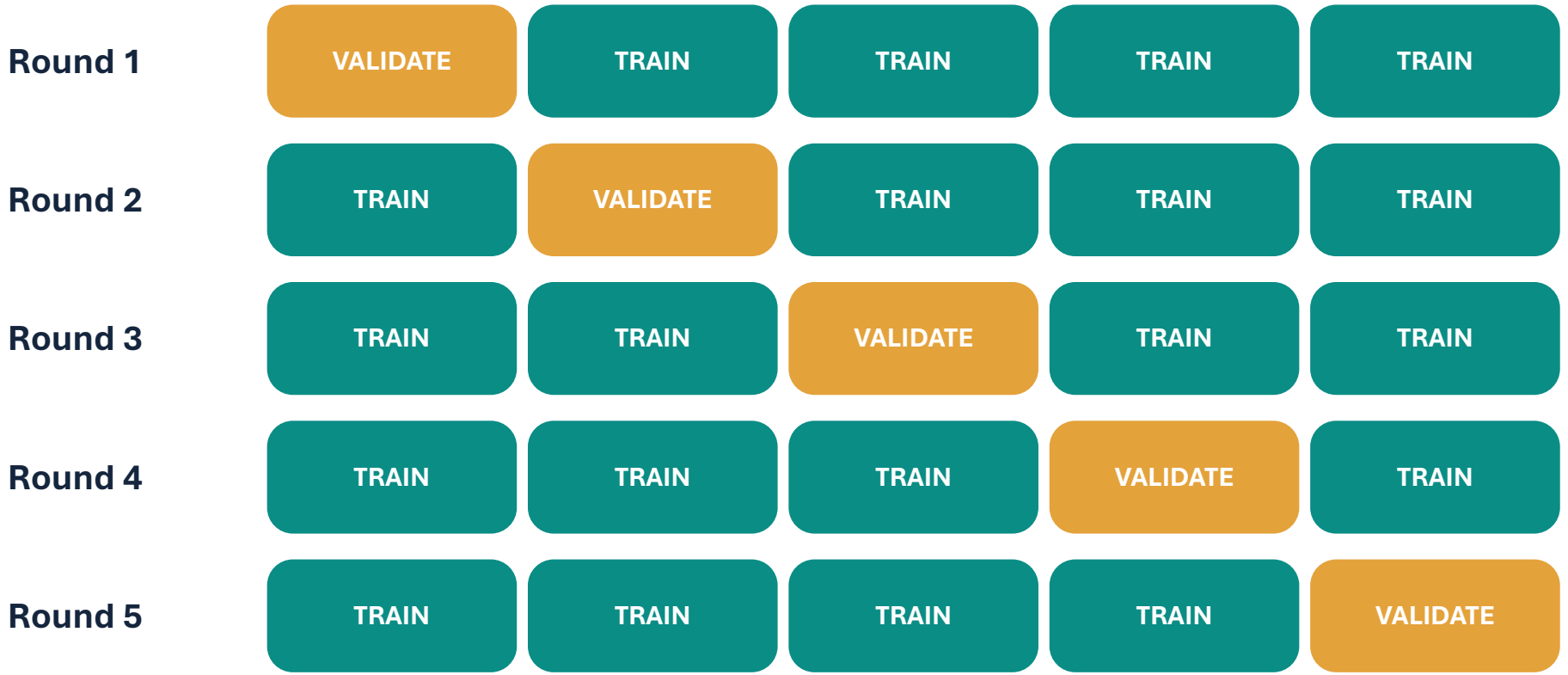
Cross-validation and hyperparameter selection

Final evaluation once

`stratify=y` • `random_state=42`

Five-fold stratified cross-validation

Each round holds out one training fold for validation.



Report mean validation score and standard deviation across folds.

Which hyperparameters shall we tune?

Parameter	Candidates	Meaning
knn__n_neighbors	1, 3, 5, ..., 21	Number of neighbors
knn__weights	uniform / distance	Equal or distance-weighted votes
knn__p	1 / 2	Manhattan / Euclidean

11
K values

× 2
Voting choices

× 2
Distance choices

= 44
Total models

GridSearchCV: The essential Python code

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid = GridSearchCV(
    estimator=knn_pipeline,
    param_grid={"knn__n_neighbors": range(1, 22, 2),
                "knn__weights": ["uniform", "distance"],
                "knn__p": [1, 2]},
    scoring="accuracy", cv=cv, refit=True
)
grid.fit(X_train, y_train)
```

How to interpret CV results

`mean_test_score`	Mean fold validation accuracy
`std_test_score`	Spread of validation scores
`mean_train_score`	Mean performance on fold train data
`rank_test_score`	Rank across parameter combinations
`best_params_`	Winning hyperparameters
`best_score_`	Best mean CV validation accuracy

“test_score” inside CV ≠ the final hold-out test score.

Observed results from the executed notebook

K = 5

Selected neighbors

p = 1

Manhattan distance

uniform

Voting weights

97.4%

Mean 5-fold CV accuracy

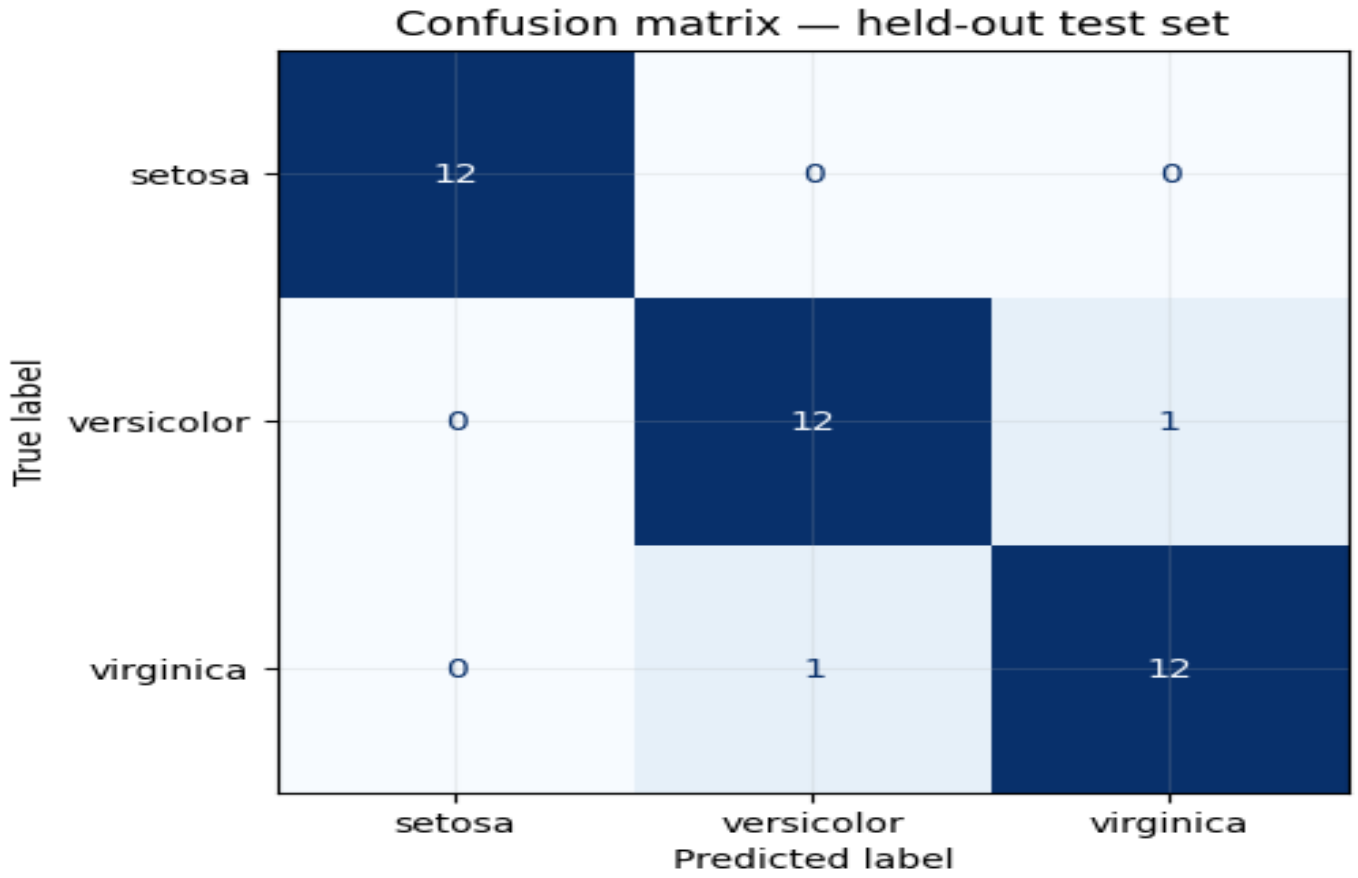
94.7%

Held-out test accuracy

The winning model did not outperform the K=5 baseline's CV score; tuning verified the choice.

Final evaluation: 38 untouched Iris examples

The confusion matrix reveals *which* classes are confused.



36 / 38

Correct predictions

94.7%

Held-out test accuracy

Precision • Recall • F1